

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	Onsdag 2022-03-14 08:00-13:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt minst hälften av poängen på B-delen och att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. <i>När du skapar projektet i IntelliJ eller NetBeans måste du kontrollera att projektet hamnar under VPROVXXXX/H, ditt tentamenskonto. Alla lösningar skrivs i ett och samma projekt, men varje uppgift ska ligga i ett separat paket, med namnen uppg1, uppg2, ...</i> Innan du avslutar tentamen: Kontrollera att du verkligen sparar dina lösningar under VPROVXXXX/H

För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent,
t.ex. "AltGr" + "{" (7:ans tangent).

Del A, Teoriuppgifter

Besvara frågorna kort men koncist, på separat papper. Motivera alltid dina svar. Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

1. Vad innebär begreppet hög kohesion inom objektorienterad programmering. Varför bör objektorienterad kod ha hög kohesion?
(2 p)
2. a) Förklara vad satsen "a = b" innebär om a och b är referensvariabler. Rita gärna.
b) Förklara vad en "garbage collector" är och vilket potentiellt problem den kan lösa i a).
(2 p)
3. Vad innebär, i objektorienterad programmering, begreppet polymorfism? Ge ett enkelt exempel på polymorf kod. Var tydlig med vad förutsättningarna är i ditt exempel.
(2 p)
4. Vad är ett interface i Java och vad får ingå i ett interface? Ange en konkret situation där det finns en poäng att deklarera ett interface.
(2 p)
5. a) Förklara hur nyckelorden "throw" respektive "throws" används i språk som Java.
b) Vad är poängen med att ha olika (sub-) typer av exceptions?
(2 p)
6. I ett elektroniskt nyhetssystem finns nyhetsgrupper och artiklar. En grupp kan innehålla artiklar men även andra grupper. Både grupper och artiklar har en titel. En artikel innehåller en text. Hela nyhetssystemet är själv en nyhetsgrupp. Det skall finnas en metod find(String title) som returnerar den första grupp eller artikel som hittas med den givna titeln. Rita ett klassdiagram med arv, associationer med multipliciteter, attribut och find-metoden som modellerar systemet.
(2p)
7. En stack är en datastruktur där objekt kan läggas till och tas bort (metoderna push och pop). För en stack gäller kontraktet att vid borttagande av objekt är det alltid det objekt som senast las till som tas bort ("last in, first out").
När man implemeterar en klass för en stack kan man utgå från en lista, t.ex. ArrayList, och antingen skapa stack-klassen via komposition eller via arv.
a) Förklara vad komposition resp. arv innebär i detta specifika fall.
b) Vilket alternativ är att föredra, med hänsyn taget till kontraktet ovan? Motivera.
(2 p)
8. I vilken situation används nyckelordet "synchronized" och vad är syftet med användandet?
(2 p)

Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg1", "uppg2" etc. Kontrollera att du verkligen sparar dina lösningar på H.

Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på P:/Java/api. *Alla klasser ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha en konstruktör som initierar alla datamedlemmar, access-metoder samt en omdefinierad version av toString.*

1. Budgivning

För att hantera budgivningar vid auktioner ska du skriva ett antal klasser som representerar data och logik för detta: Auction, Bid och User.

Klassen User representerar en användare i systemet, t.ex. en säljare eller en budgivare. User har ett namn, ett id samt en e-postadress. Endast e-postadressen ska kunna ändras.

E-postadressen ska representeras av en egen klass, Email. Klassens konstruktör ska vara privat. För att få tillgång till ett nytt e-post-objekt ska det finnas en klassmetod, getInstance, som tar en textsträng som argument, och returnerar ett nytt objekt. Innan objektet skapas ska metoden kontrollera om indata, textsträngen, är en korrekt e-postadress; om så inte är fallet ska ett IllegalArgumentException kastas. Det räcker att kontrollera om texten innehåller ett "@".

Klassen Bid representerar ett Bud, med budgivare, av typen User, belopp samt tid för budet, LocalDateTime. Objekt av typen Bid ska vara "immutable" (oföränderliga).

Klassen Auction representerar en vara som säljs via budgivning. Klassen har en säljare, av typen User, namnet på varan som säljs, en sluttid (dessa ska inte kunna ändras) samt en lista med de bud som givits.

Det ska förutom meningsfull konstruktör och liknande finnas följande metoder:

- addBid, med argument för budgivare och belopp, som skapar ett bud och lägger det i listan. Om budet är lägre än det tidigare högsta, eller sluttiden passerats, ska ett IllegalBidException kastas (du skriver själv denna klass).
- getHighestBid, som returnerar det högsta budet.
- isClosed, returnerar true om sluttiden passerat.
- getBids, som returnerar en kopia av listan med bud.

Skriv sedan en main-metod som skapar objekt av typerna ovan och demonstrerar funktionaliteten. (4 p)

2. Olika typer av konton

Gemensamt för olika typer av bankkonton är att de har en innehavare, av typen User, ett kontonummer samt ett saldo. Innehavare och kontonummer ska inte gå att ändra efter att ett konto skapats.

a) Skriv en *abstrakt* klass, Account, som representerar allt som är gemensamt för alla olika typer av konton. Klassen ska ha lämpliga datamedlemmar samt följande metoder:

- En meningsfull konstruktör.
- deposit(...), en insättningsmetod
- En *abstrakt* metod för uttag, withdraw(...)

Både deposit och withdraw ska kasta `IllegalArgumentException` om indata till metoderna är inkorrekt (t.ex. Om beloppet är negativt).

Klassen ska även implementera interfacet `Comparable`, och rangordna efter kontonumret (som anses unikt). `Object.equals` ska omdefinieras så att den returnerar `true` när `compareTo` returnerar 0.

Skriv sedan subklasserna `SavingsAccount` och `CreditAccount`.

- `SavingsAccount` får inte ha negativt saldo och håller reda på hur många uttag som har gjorts. Om antalet uttag är fler än fem tas en uttagsavgift på 10 kr ut för varje uttag därefter.
- `CreditAccount` har en kreditgräns, unik för varje objekt. Kreditgränsen anges då objektet skapas och vid uttag får inte uttaget överskrida summan av saldo och kreditgräns.
- För båda sub-klasserna ska en omdefiniering av metoden `withdraw` finnas. Notera att villkoret för när exception ska kastas ser olika ut i subklasserna.

Skriv ett main där du skapar en lista med referenser av bas-typen `Account`. Skapa sedan objekt av subklasserna, lägg dessa i arrayen/listan och demonstrera funktionaliteten.

(4 p)

3. Strategi för rangordning

Skriv ett generiskt interface, `Ranker`, för att rangordna objekt av godtycklig typ. Interfacet ska ha en metod, `rank`, som tar två objekt som argument och returnerar 0 om objekten anses lika, negativt om det första objektet anses komma före det andra, och positivt i övriga fall.

Skriv sedan, i en separat klass, en klassmetod för att sortera objekt. Metoden ska ha två argument, en array av objekt samt en implementation av `Ranker`. Sorteringsmetoden ska vara generisk - deklarera den som `"public static <T> void sort( ... )"`. Du kan sortera med bubble-sort eller med den sorteringsalgoritm som beskrivs på sista sidan.

Implementera sedan interfacet `Ranker` i två fall, en som rangordnar objekt av typen `User` efter namn, samt en version som rangordnar samma typ av objekt efter e-postadress (i båda fallen alfabetiskt). Skapa en array av `User`-objekt och låt sorteringsmetoden sortera arrayen med respektive `Ranker`.

(4 p)

4. Cirkulär kö

En kö, `Queue`, är en datastruktur där element läggs till i ena änden, `enqueue`, och plockas ut i andra änden, `dequeue`, dvs. när ett element tas bort från kön är det alltid det som legat längst i kön.

Skriv en generisk klass, `Queue`, där objekt lagras i en privat array. Implementera klassen så att nya element läggs efter det senaste, mot slutet av arrayen, och element tas bort från början av arrayen. När ett nytt objekt läggs till, och slutet på arrayen nåtts, placeras objektet istället i början av arrayen (om denna är ledig). I denna mening är kön cirkulärt implementerad.

Om inga lediga positioner existerar när ett nytt element läggs till låter du en intern metod utöka storleken på arrayen.

Förutom meningsfull konstruktor och `toString` ska klassen ha följande metoder:

- `size`, som returnerar aktuellt antal element (inte detsamma som arrayens storlek),
- `enqueue`, som lägger till ett nytt element sist i kön,

- dequeue, som returnerar, och tar bort, det element som legat längst tid i kön,
- getAll, som returnerar en array med de element som finns i kön - denna array ska vara en kopia av den interna arrayen - storleken bestäms av antalet objekt, det objekt som köat längst ska ligga på index 0.

Skriv en main-metod där du skapar en kö för String-objekt och demonstrerar funktionaliteten.

(4 p)

Urvalssortering (selection sort)

Pesudokoden nedan visar sortering av heltal. I din implementation använder du istället en Ranker vid jämförelsen av värden (istället för "<").

```
for (i: [0..length-1]) {
    index = i; // index for min value
    // find the min value among the remaining values
    for(j: [i+1..length]) {
        if(A[j] < A[index])
            index = j;
    }
    // swap
    temp = A[i];
    A[i] = A[index];
    A[index] = temp;
}
```